RED & WHITE SKILL EDUCATION

DEEP LEARNING

PR 6

Project Definition

Total Marks: 10

Field	Detail
Institute	Red & White Skill Education
Subject	Deep Learning
Project	Practical Report 6 (PR 6) — Opening the Transformer: Attention, Encoder, Decoder and Inference with Pretrained Models
Total Marks	10
Project Duration	2 Days · 2 Hours per Day = 4 Hours
Approach	Use pretrained Hugging Face models. NO model training or fine-tuning is required — the practical is about understanding and inspecting how a Transformer works, and using it.
Data	A 200-review sample of the public IMDB dataset (loaded with the datasets library) plus 3 real text passages chosen by the student
Framework	Hugging Face transformers + PyTorch + datasets (Python 3.9+, Jupyter Notebook or Google Colab)
Hardware	Everything runs on a normal laptop CPU. A free Google Colab session is optional.

Lecture Topic Coverage Map — Transformers Module

Task	Lecture Topics Covered (Transformers Module)
Task 1	Introduction to Transformers, History, Impact and Disadvantages; first look at the Hugging Face pipeline API and tokenizer
Task 2	What is Self-Attention, Scaled Dot-Product Attention, Self-Attention vs Luong Attention
Task 3	Multi-Head Attention, Positional Encoding, Layer Normalization
Task 4	Transformer Encoder Architecture, Masked Self-Attention
Task 5	Cross-Attention, Transformer Decoder Architecture, Transformer Inference
Task 6	Mini application, evaluation of a pretrained model and the Disadvantages of Transformers observed in practice

§ 1 · Models, Data & Environment

This is the final and simplest practical of the Deep Learning course. Instead of training anything, you will download small pretrained Transformer models from the Hugging Face Hub and open them up: run them, inspect their attention weights, rebuild the key equations yourself in NumPy, and check that your version agrees with the real model. Three small models are enough to cover every lecture topic, because together they represent the three Transformer families: encoder-only, decoder-only and encoder-decoder.

Hugging Face Model ID	Architecture	Approx. Size	Used For
distilbert-base-uncased-finetuned-sst-2-english	Encoder-only (6 layers, 12 heads)	about 270 MB	Sentiment analysis, attention inspection, encoder anatomy, positional and layer-norm study
gpt2	Decoder-only (12 layers, 12 heads)	about 550 MB	Masked (causal) self-attention, text generation, manual inference loop
t5-small	Encoder-decoder (6 + 6 layers, 8 heads)	about 240 MB	Cross-attention, translation and summarisation

Equivalent models: You may substitute a similar small model (for example bert-base-uncased or distilgpt2) if one of the above is unavailable, but you must state the exact Hugging Face model ID for every experiment in your README. Do not use models larger than about 1 GB.

Field	Detail
Install	pip install transformers torch datasets matplotlib seaborn numpy pandas
Hugging Face account	Not required — all three models and the IMDB dataset are public
Internet	Needed only for the first download of each model; models are cached locally afterwards
Dataset loading	from datasets import load_dataset · ds = load_dataset('imdb', split='test').shuffle(seed=42).select(range(200))
Dataset content	200 real movie reviews with labels (0 = negative, 1 = positive); several reviews are longer than the 512-token limit of the model, which is used in Task 6
Getting attention weights	Load models with attn_implementation='eager' (for example AutoModel.from_pretrained(name, attn_implementation='eager')) and pass output_attentions=True. The faster default implementation does not return attention weights in recent versions of transformers.
Reproducibility	Set torch.manual_seed(42) and np.random.seed(42) at the top of the notebook; call model.eval() before every inference

§ 2 · Project Tasks — What to Do

Complete ALL tasks in a single notebook named DL_PR6.ipynb. Begin every task with a Markdown cell stating the task title, a one-sentence objective and the expected output. Keep your Markdown explanations short and in your own words (3-6 sentences per concept is enough). Your goal is to understand WHY each part of the Transformer exists, not to write long code.

Task 1: Introduction, History, Pipelines & Tokenizer

#	Action	Details / Configuration
1.1	**Install and first pipeline**	Install the libraries. Run: from transformers import pipeline. clf = pipeline('sentiment-analysis', model='distilbert-base-uncased-finetuned-sst-2-english'). Call clf on 5 sentences you write (positive, negative, and one tricky sentence such as a negation). Print the label and score for each. Markdown: one pipeline call hides three steps — tokenisation, model forward pass, post-processing.
1.2	**Introduction & History — Markdown**	Write a short timeline table (year, model or paper, why it mattered) covering at least: 2014-15 sequence-to-sequence with attention (Bahdanau / Luong), 2017 'Attention Is All You Need' (Vaswani et al.), 2018 BERT and GPT, 2019 GPT-2 / T5 / DistilBERT, 2020 and later large language models and Vision Transformers. Add 2-3 sentences on what problem with RNNs / LSTMs (Module 9: sequential processing, short memory, slow training) the Transformer solved.
1.3	**Impact & Disadvantages — Markdown**	Write two short lists. Impact: examples of tasks transformed by Transformers (translation, search, chat assistants, code completion, image models, protein structure). Disadvantages: quadratic attention cost in sequence length, fixed context window, large compute and memory needs, need for huge data, hallucination in generative models, opaque behaviour. You will measure two of these yourself in Task 6.
1.4	**Tokenizer in action**	from transformers import AutoTokenizer. tok = AutoTokenizer.from_pretrained('distilbert-base-uncased-finetuned-sst-2-english'). Tokenise 'Transformers are surprisingly easy to understand.' Print tok.tokenize(text), the input_ids and the attention_mask. Show a rare or long word being split into sub-word pieces. Markdown: what do the special tokens [CLS] and [SEP] do, and why sub-words instead of whole words?
1.5	**Batching and padding**	Tokenise two sentences of different length together with padding=True and return_tensors='pt'. Print input_ids and attention_mask side by side. Markdown: the attention_mask marks real tokens (1) versus padding (0) so the model ignores padding — you will see the effect in Task 4.
1.6	**Model anatomy at a glance**	Load the model with AutoModelForSequenceClassification and print model.config (number of layers, attention heads, hidden size, vocabulary size, maximum position embeddings). Count total parameters with sum(p.numel() for p in model.parameters()). Print the first 15 names from model.named_parameters(). Add a small table listing the numbers you found; you will explain each of them in Tasks 2-4.

Task 2: Self-Attention & Scaled Dot-Product Attention

#	Action	Details / Configuration
2.1	**Self-attention idea — Markdown**	Explain in plain words: every token is turned into three vectors — a Query (what am I looking for), a Key (what do I offer) and a Value (what I pass on). A token's new representation is a weighted average of all Value vectors, where the weights come from how well its Query matches every Key. Mention that the whole sequence is processed in parallel, unlike an RNN.
2.2	**Scaled dot-product attention from scratch**	In NumPy: take a 4-token sequence with d_model = 8 (random X with a fixed seed). Create Wq, Wk, Wv (8×8) at random. Compute Q = X@Wq, K = X@Wk, V = X@Wv. Implement scores = Q @ K.T / np.sqrt(d_k), attention weights = softmax(scores, axis=-1) (write your own stable softmax), and output = weights @ V. Print every intermediate shape. Plot the 4×4 attention-weight matrix as a heatmap and confirm every row sums to 1.
2.3	**Why divide by sqrt(d_k)?**	Repeat the score calculation with d_k = 4 and d_k = 256 using random Q and K, once WITH and once WITHOUT the 1/sqrt(d_k) scaling. For each of the four cases print the maximum softmax probability averaged over rows (and the entropy if you can). Markdown: explain that dot products grow in magnitude with d_k, pushing softmax into a saturated region where one weight is almost 1 and gradients almost vanish — scaling keeps the variance near 1.
2.4	**Self-attention vs Luong attention**	Implement Luong 'dot' attention in NumPy: 5 encoder hidden states (5×8) and one decoder hidden state (8,). scores = encoder_states @ decoder_state, weights = softmax(scores), context = weights @ encoder_states. Print the weights. Then add a Markdown comparison table: Luong attention vs self-attention on — where Q, K, V come from, what is attended over (decoder state looks at encoder states vs every token looks at every token), whether it needs an RNN, parallelism, and where it sits in the architecture.
2.5	**Attention is order-blind — demonstration**	Using your function from 2.2, shuffle the 4 rows of X with a fixed permutation, run self-attention again, and show that the OUTPUT rows are simply the same rows in the shuffled order (np.allclose after re-permuting). Markdown: self-attention has no notion of position, which is exactly why Task 3 needs positional encoding.
2.6	**Verify against a real model**	Load DistilBERT with attn_implementation='eager', run one sentence with output_hidden_states=True and output_attentions=True. For layer 0, head 0 recompute the attention matrix yourself: take hidden_states[0] as input, apply the layer's q_lin and k_lin (print(model) to see the exact attribute names), split into 12 heads of size 64, compute softmax(Q K^T / sqrt(64)) after adding the padding mask if any, and compare with attentions[0][0, 0] using np.allclose (tolerance 1e-4). Print the maximum absolute difference. Markdown: what does a successful match prove?

Task 3: Multi-Head Attention, Positional Encoding & Layer Normalization

#	Action	Details / Configuration
3.1	**Multi-head attention — Markdown**	Explain why one attention pattern is not enough: with h heads, d_model is split into h smaller subspaces (for DistilBERT 768 = 12 × 64), each head learns its own Q, K, V projections and attention pattern, then the heads are concatenated and mixed by an output projection. State that the total computation stays about the same as one big head.
3.2	**Extract and plot real attention heads**	Run the sentence 'The animal did not cross the street because it was too tired.' through DistilBERT with output_attentions=True. The result is

		a tuple of 6 tensors, each shaped (batch, 12, tokens, tokens). Print the shapes. Plot a 2×3 grid of heatmaps for 6 different heads (mix of layers) with token labels on both axes. Markdown: describe at least two different behaviours you see (for example a head attending to the previous token, a head attending to [CLS] / [SEP], a head linking 'it' with a noun).
3.3	**Head diversity measurement**	For layer 0 and the last layer, compute the average attention entropy of each of the 12 heads (low entropy = focused, high entropy = spread out). Plot as a bar chart per layer. Markdown: do early and late layers behave differently? Keep your claims modest — this is one sentence only.
3.4	**Sinusoidal positional encoding from scratch**	Implement the original Transformer encoding in NumPy: PE(pos, 2i) = sin(pos / 10000^(2i/d_model)), PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model)) for 50 positions and d_model = 128. Plot it as a heatmap (positions on the y-axis, dimensions on the x-axis). Markdown: why sines and cosines of different frequencies (each position gets a unique pattern; relative offsets are easy to learn).
3.5	**Position makes identical words different**	Take a single word embedding vector (random, d_model = 128) and place it at positions 3 and 20 of a sentence by adding the matching rows of your PE. Print the cosine similarity of the two results compared with the similarity without PE (which is 1.0). Combine with Task 2.5: explain how adding PE removes the order-blindness.
3.6	**Learned positional embeddings in a real model**	Inspect model.distilbert.embeddings.position_embeddings.weight (shape 512 × 768). Compute the cosine-similarity matrix of the first 64 positions and plot it as a heatmap. Markdown: BERT-style models LEARN their position vectors instead of using sinusoids; note the maximum of 512 positions as the origin of the context-length limit. Add one sentence noting that T5 uses relative position biases instead (you will meet it in Task 5).
3.7	**Layer normalization from scratch**	In NumPy implement LayerNorm: for each token vector x (length d), y = gamma * (x - mean(x)) / sqrt(var(x) + eps) + beta with the mean and variance taken over the FEATURE dimension of that single token. Run it on a random (4 tokens × 8) matrix, print per-token mean (about 0) and standard deviation (about 1). Compare with torch.nn.LayerNorm(8) (gamma = 1, beta = 0) using torch.allclose.
3.8	**LayerNorm vs BatchNorm — Markdown and check**	Run the same matrix through a BatchNorm that normalises each FEATURE across the 4 tokens and show that per-token statistics are NOT normalised. Write a short comparison: what is normalised over, dependence on batch size, behaviour with variable-length sequences and padding, and why Transformers use LayerNorm (compare with Batch Normalization from PR 2). Finally print the LayerNorm weights of DistilBERT's embeddings layer (shape 768) and state what gamma and beta are.

Task 4: Transformer Encoder Architecture & Masked Self-Attention

#	Action	Details / Configuration
4.1	**Encoder architecture — Markdown**	Draw or describe the encoder block: input embeddings + positional information → N stacked blocks, each = multi-head self-attention → residual connection and layer norm → position-wise feed-forward network (expand, non-linearity, project back) → residual connection and layer norm. State what each of the 6 layers of DistilBERT contains, and that 'encoder' means every token can see every other token in both directions.

4.2	**Print and read the real encoder**	print(model.distilbert) for DistilBERT. Identify and write down (in a small table) the module names for: token embedding, position embedding, embedding LayerNorm, one attention block (q_lin, k_lin, v_lin, out_lin), the feed-forward block (lin1, lin2, activation), and the two layer norms per block. Note the feed-forward expansion from 768 to 3072 and back.
4.3	**Where do the parameters live?**	Group model parameters by name into: embeddings, attention, feed-forward, layer norms, classification head. Compute each group's parameter count and share of the total. Show the result as a bar or pie chart. Markdown: which component holds most parameters, and what does that tell you about where the model stores knowledge?
4.4	**Use the encoder: fill-mask**	Use pipeline('fill-mask', model='distilbert-base-uncased') on three sentences containing [MASK] (for example 'The capital of France is [MASK].'). Print the top-3 predictions with scores. Markdown: the model uses words on BOTH sides of the mask — this is the bidirectional encoder behaviour.
4.5	**Bidirectional attention heatmap**	Plot the full attention matrix (average over heads of the last layer) of DistilBERT for one sentence. Confirm that there are non-zero weights both above and below the diagonal. Markdown: explain that this is NOT allowed in a language model that must predict the next word — leading to masked self-attention.
4.6	**Masked (causal) self-attention from scratch**	In NumPy, build a causal mask for 6 tokens: a matrix with -infinity above the diagonal and 0 on and below it. Add it to the score matrix from your Task 2.2 style function BEFORE the softmax. Plot the resulting attention matrix and confirm it is lower-triangular with rows summing to 1. Markdown: why add -infinity before softmax rather than zeroing weights afterwards.
4.7	**Causal mask in a real decoder**	Load GPT-2 with attn_implementation='eager', run one sentence with output_attentions=True and plot the attention matrix of one head from the last layer. Confirm numerically that every entry above the diagonal is (almost) zero, e.g. np.triu(attn, k=1).max() < 1e-6. Compare the picture with your bidirectional plot from 4.5.
4.8	**Padding mask vs causal mask**	Tokenise a batch of two sentences of different length (with padding) and run it through DistilBERT with output_attentions=True. Show that the attention weights received by the padding positions are (almost) zero. Markdown table: padding mask vs causal mask — purpose, shape pattern, which model families use each.

Task 5: Cross-Attention, Decoder Architecture & Inference

#	Action	Details / Configuration
5.1	**Decoder architecture — Markdown**	Describe the original Transformer decoder block: masked multi-head self-attention → add and norm → cross-attention over the encoder output → add and norm → feed-forward → add and norm, followed by a linear layer and softmax over the vocabulary. Then explain the three model families: encoder-only (DistilBERT / BERT), decoder-only (GPT-2, which drops cross-attention), encoder-decoder (T5, which keeps both).
5.2	**Cross-attention — Markdown**	In cross-attention the Queries come from the DECODER while the Keys and Values come from the ENCODER output. Explain in your own words what this lets the decoder do (look at the source sentence while writing the target sentence) and relate it to Luong attention from Task 2.4.
5.3	**Run an encoder-decoder model**	Use pipeline('translation_en_to_fr', model='t5-small') on 3 English sentences and print the French output. Then use

		pipeline('summarization', model='t5-small') on one real paragraph (about 150-300 words, for example a news article you paste in) with max_length=60 and min_length=20. Print input length and summary length in words.
5.4	**Extract and plot cross-attention**	Load T5ForConditionalGeneration and T5Tokenizer for t5-small (attn_implementation='eager'). Encode 'translate English to French: The house is small.' and tokenise the expected output 'La maison est petite.' as labels. Run model(input_ids=..., labels=..., output_attentions=True) and read outputs.cross_attentions. Take the last decoder layer, average over the 8 heads and plot a heatmap: rows = output tokens, columns = input tokens. Markdown: which input words does each output word attend to? Is the alignment sensible (for example 'maison' to 'house')?
5.5	**Inference is autoregressive — manual loop**	With GPT-2, write your own greedy decoding loop: start from the prompt 'Artificial intelligence will', and for 20 steps call model(input_ids).logits[:, -1, :], pick argmax, and append the new token. Decode and print the text. Then call model.generate(input_ids, max_new_tokens=20, do_sample=False) and check that the output is IDENTICAL. Markdown: explain that a decoder generates ONE token at a time and feeds it back as input.
5.6	**Decoding strategies**	With the same prompt, generate 40 tokens using: (a) greedy, (b) beam search (num_beams=5, early_stopping=True), (c) sampling with temperature=0.7, (d) sampling with temperature=1.5, (e) top_k=50 with top_p=0.9. Present the five outputs in a table. For (c) and (d) run twice with different seeds. Markdown: how do temperature, top-k and top-p change randomness and repetition?
5.7	**KV-cache speed-up**	Time generating 100 new tokens with GPT-2 using model.generate(..., use_cache=True) versus use_cache=False (average of 3 runs each, on CPU is fine). Report both times and the speed-up. Markdown: the key and value vectors of earlier tokens do not change, so they can be cached instead of recomputed at every step — this is the main speed trick in Transformer inference.
5.8	**Inference summary table — Markdown**	Compare the three model families in one table: DistilBERT, GPT-2, T5-small — columns: family, attention type(s) used, can it generate text, position scheme (learned / relative), typical tasks, example from this notebook.

Task 6: Mini Application, Evaluation & Disadvantages in Practice

#	Action	Details / Configuration
6.1	**Evaluate the pretrained sentiment model**	Load the 200 IMDB reviews. Run the DistilBERT sentiment pipeline on all of them with truncation=True and max_length=512 (use batch_size=16). Map labels to 0/1 and compute accuracy, precision, recall, F1 and a confusion matrix. Note that you did NOT train anything — the model was already fine-tuned on a different dataset.
6.2	**Effect of truncation length**	Repeat the evaluation with max_length = 64, 128, 256 and 512. Plot accuracy against max_length and print the time taken for each run. Markdown: how much accuracy is lost when the context window is small, and what does it cost in time? Count how many of the 200 reviews exceed 512 tokens and therefore lose text even at the maximum setting.
6.3	**Cost grows with sequence length**	Measure the time of a single DistilBERT forward pass (batch size 1, average of 10 runs after a warm-up run) for input lengths 32, 64, 128, 256 and 512 tokens (build inputs by repeating text and truncating). Plot

		time against length. Markdown: attention compares every token with every other token, so cost grows roughly with the SQUARE of the length — relate this to the 512-token context limit. State honestly if your CPU measurements look closer to linear at small sizes.
6.4	**Hallucination check**	Give GPT-2 three prompts that ask for facts (for example 'The capital of Australia is', 'The author of the novel Pride and Prejudice is', 'The inventor of the telephone was'). Generate 30 tokens for each with greedy decoding. Mark each output as correct, partly correct or wrong. Markdown: what does this tell you about generative models as sources of facts? (This is the hallucination disadvantage from Task 1.3.)
6.5	**Build the mini application**	Write a function text_toolkit(text) that returns a dictionary with: (a) sentiment label and score (DistilBERT), (b) a short summary (t5-small, max_length = 60), (c) the French translation of the summary (t5-small). Test it on 3 real passages of your choice (one movie or product review, one news paragraph, one passage of your own) and display the results as a pandas DataFrame. Handle empty or very long inputs gracefully with truncation and a clear message.
6.6	**Reflection — Markdown**	Write a final section titled 'What I learned about Transformers' covering: (a) which part of the Transformer you now understand best and which is still unclear; (b) one thing the real model did that matched the theory (for example verified attention matrix, causal mask) and one thing that surprised you; (c) the two disadvantages you observed in this notebook with the numbers you measured; (d) one real-world application you would build with these models and which model family it needs.
6.7	**Notebook quality**	Every task has an opening Markdown cell. All plots have titles, axis labels and token labels where relevant. Seeds are set at the top. Restart the kernel and run all cells with zero errors. Export to HTML. Create requirements.txt listing the library names and the versions you actually used (pip freeze | grep -E 'transformers|torch|datasets').

§ 3 · Transparent Marking Scheme (10 Marks)

Total Marks: 10 — distributed across 3 components as follows:

Component	Marks	Weight	Deliverable
A. Technical Completion (Jupyter Notebook .ipynb)	5 Marks	50%	Completed notebook — all 6 tasks, Restart & Run All clean
B. Recorded Video Explanation (Face + Screen)	3 Marks	30%	5–10 min MP4/MOV: face + screen, verbal concept explanations
C. GitHub Repository + README.md	2 Marks	20%	Public repo with .ipynb + .html + plots + README + video link
TOTAL	**10 Marks**	**100%**	

Component A: Technical Completion (5 Marks)

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
A1	**0.5**	Pipeline run on 5 sentences; history timeline, impact and disadvantages written; tokenizer output with special tokens and sub-words shown; padded batch with attention_mask; model.config table with layers, heads, hidden size and parameter count	Pipeline, tokenizer and config done; timeline or disadvantages Markdown thin	Pipeline and tokenizer only; no background Markdown	Pipeline run once; no explanation	Not done
A2	**1.5**	Scaled dot-product attention implemented in NumPy with shapes printed and row-sum check; scaling experiment with and without 1/sqrt(d_k) at two values of d_k; Luong dot attention implemented with	All pieces done; verification against the real model or Luong comparison incomplete	Scaled dot-product attention implemented; no scaling experiment or Luong comparison	Only a Markdown description or only a library call	None

		comparison table; order-blindness demonstration; layer-0 head-0 attention recomputed from DistilBERT weights and matched with np.allclose				
A3	**1.0**	Real attention heads plotted (6 heads with token labels) with two behaviours described; sinusoidal positional encoding heatmap; same-word-different-position demonstration; learned position embedding similarity plot; LayerNorm from scratch matched with torch.nn.LayerNorm; LayerNorm vs BatchNorm comparison	Heads, PE and LayerNorm done; one of the comparison demonstrations or the learned-position plot missing	Heads plotted and PE implemented; LayerNorm only described	One topic done partially	None
A4	**1.0**	Encoder structure printed and tabulated; parameter distribution chart; fill-mask on 3 sentences; bidirectional vs causal attention heatmaps; causal mask built in NumPy and confirmed in GPT-2 with np.triu check; padding mask demonstration; cross-attention heatmap from t5-small with alignment discussion; manual greedy loop identical to generate(); five decoding strategies	Most items done; two sub-items missing (for example cross-attention plot or KV-cache timing)	Encoder and causal-mask parts done; no cross-attention or inference experiments	Only pipelines run; no inspection of attention	None

		compared; KV-cache timing				
A5	**1.0**	Pretrained sentiment model evaluated on 200 reviews with full metrics; truncation-length study with time; forward-pass time vs length plot with honest interpretation; hallucination check with graded outputs; text_toolkit() working on 3 real passages with graceful handling of bad input; reflection section; seeds set; Restart & Run All clean	Evaluation, toolkit and one limitation experiment done; reflection or error handling missing	Evaluation and toolkit done; no limitation experiments	Toolkit only, or evaluation only	None

Subtotal A: 0.5 + 1.5 + 1.0 + 1.0 + 1.0 = 5.0 Marks

Component B: Recorded Video — Face + Screen (3 Marks)

Record a video showing your face (webcam picture-in-picture) and your notebook screen. Explain concepts verbally — do not just run cells silently.

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
B1	**1.0**	Clear audio and video; face visible via PiP throughout; heatmaps, tables and outputs clearly visible; all 6 tasks demonstrated in order	Clear audio; face mostly visible; most tasks shown	Audio unclear; face partially visible	Very poor quality, hard to follow	No video
B2	**1.5**	Verbally explains: (1) Q, K, V and WHY scores are divided by sqrt(d_k); (2) what multi-head attention adds beyond one	Explains 5–6 concepts with reasoning	Demonstrates steps without explaining theory	Reads from cells; no original explanation	No verbal explanation

		head; (3) WHY positional encoding is needed (order-blind demonstration) and the difference between sinusoidal and learned positions; (4) LayerNorm versus BatchNorm; (5) encoder versus decoder and what masked self-attention prevents; (6) cross-attention — where Q and K/V come from — using your heatmap; (7) how inference works token by token, what the KV-cache saves and one disadvantage you measured				
B3	**0.5**	5–10 min; shows the NumPy attention heatmap, a real-model attention plot, the causal mask, the cross-attention heatmap and a LIVE text_toolkit() demo on a new passage	4–5 min; most tasks covered	2–4 min; key experiments missing	Under 2 min or very incomplete	Not submitted

Subtotal B: 1.0 + 1.5 + 0.5 = 3.0 Marks

Video Requirements

Requirement	Detail
Format	MP4 or MOV, maximum 500 MB
Face visibility	Webcam picture-in-picture overlay visible throughout the full recording
Screen content	Notebook — attention heatmaps, causal mask, cross-attention plot, decoding table, live text_toolkit() demo

Duration	5 to 10 minutes
Naming convention	DL_PR6_YourName_GRID.mp4
Upload location	Google Drive (Anyone with link can view) or YouTube (Unlisted)
Link placement	Paste the video URL in your GitHub README.md under the Video section

Component C: GitHub Repository + README.md (2 Marks)

Create a public GitHub repository specifically for this project.

Criterion	Marks	Excellent (Full)	Good (75%)	Average (50%)	Poor (25%)	Zero
C1	**0.5**	Repo contains: DL_PR6.ipynb + DL_PR6.html + /plots with clearly named figures (attention_scratch.png, attention_heads.png, positional_encoding.png, causal_mask.png, cross_attention.png, truncation_accuracy.png, time_vs_length.png) + requirements.txt	Key files present; a few plots missing	Only .ipynb uploaded; no plots	Repo exists but incomplete or broken	No repo
C2	**1.0**	README: title + a table of the exact Hugging Face model IDs used + one paragraph per major concept (self-attention, multi-head, positional encoding, layer norm, masked attention, cross-attention, inference) + at least 3 plots with captions + evaluation results + the two measured disadvantages + video link	Title + model table + partial explanations and plots	Title and model table only	Blank README or single line	No README

C3	**0.5**	3+ meaningful commits, e.g. 'Add pipelines, tokenizer and model anatomy'; 'Add attention from scratch and verification against DistilBERT'; 'Add decoder, cross-attention, inference experiments and mini application'	2–3 adequate commits	1 bulk commit with all files	Unclear or auto-generated messages	No commits

Subtotal C: 0.5 + 1.0 + 0.5 = 2.0 Marks

Faculty Marking Sheet

Component	Criterion	Max	Score	Remarks
A (5)	A1. Introduction, History, Pipelines, Tokenizer & Model Anatomy	0.5		
	A2. Self-Attention, Scaling, Luong Comparison & Verification	1.5		
	A3. Multi-Head Attention, Positional Encoding & Layer Normalization	1.0		
	A4. Encoder, Masked Attention, Cross-Attention, Decoder & Inference	1.0		
	A5. Evaluation, Disadvantages, Mini Application & Reflection	1.0		
B (3)	B1. Video Quality (Audio, Face PiP, Screen Clarity)	1.0		
	B2. Verbal Explanation of 7 Core Concepts	1.5		
	B3. Duration & Completeness (5–10 min, live toolkit demo)	0.5		
C (2)	C1. Repo Structure (.ipynb + .html + /plots + requirements.txt)	0.5		
	C2. README.md Quality (model table, concepts, plots, results)	1.0		
	C3. Commit History (3+ meaningful commits)	0.5		
TOTAL		**10.0**		

Grade Bands

Score	Grade	Descriptor
9.0 – 10.0	A+ Outstanding	All 6 tasks complete; attention recomputed from scratch and verified against the real model; cross-attention alignment explained; disadvantages measured honestly; confident verbal explanations; portfolio-ready notebook
8.0 – 8.9	A Excellent	All tasks functional; verification and inference experiments done; strong explanations; minor gaps in plots or measurements
7.0 – 7.9	B+ Very Good	Core attention, encoder and decoder tasks done; verification or one inference experiment missing
6.0 – 6.9	B Good	Self-attention and multi-head parts complete; decoder / cross-attention only partly done
5.0 – 5.9	C Satisfactory	Pipelines run and basic attention implemented; limited inspection and analysis
Below 5.0	D Needs Improvement	Mostly pipeline calls with no inspection of attention; rework required

§ 4 · Submission Checklist

Tick every item below before submitting. All items are required for full marks.

- ☐ transformers, torch, datasets, matplotlib, seaborn, numpy, pandas installed; seeds set at the top of the notebook
- ☐ Sentiment pipeline run on 5 self-written sentences with labels and scores
- ☐ History timeline table (2014-15 to present) and Markdown on what problem Transformers solved over RNN / LSTM
- ☐ Impact and Disadvantages lists written
- ☐ Tokenizer output shown: tokens, input_ids, attention_mask, special tokens and a sub-word split
- ☐ Padded batch of two sentences with attention_mask printed
- ☐ model.config values, parameter count and parameter names tabulated
- ☐ Self-attention idea Markdown (Query, Key, Value)
- ☐ Scaled dot-product attention implemented in NumPy; shapes printed; heatmap plotted; rows sum to 1
- ☐ Scaling experiment: with and without 1/sqrt(d_k) for two values of d_k; explanation of softmax saturation
- ☐ Luong dot attention implemented in NumPy; comparison table with self-attention
- ☐ Order-blindness demonstration: shuffled input gives shuffled output
- ☐ Layer-0 head-0 attention recomputed from DistilBERT q_lin / k_lin and matched with np.allclose
- ☐ Multi-head attention Markdown; real attention shapes printed
- ☐ 2×3 grid of real attention heads with token labels; two behaviours described
- ☐ Attention entropy per head for layer 0 and last layer plotted
- ☐ Sinusoidal positional encoding implemented and plotted as heatmap
- ☐ Same word at two positions: cosine similarity with and without PE
- ☐ DistilBERT learned position embeddings: cosine-similarity heatmap of first 64 positions; 512 limit noted
- ☐ LayerNorm implemented in NumPy and matched with torch.nn.LayerNorm
- ☐ LayerNorm vs BatchNorm comparison with demonstration; DistilBERT LayerNorm gamma / beta shapes printed
- ☐ Encoder block described; DistilBERT structure printed; module-name table completed
- ☐ Parameter distribution by component (embeddings, attention, feed-forward, layer norms, head) charted
- ☐ fill-mask pipeline on 3 sentences with top-3 predictions
- ☐ Bidirectional attention heatmap (DistilBERT) and causal attention heatmap (GPT-2) compared
- ☐ Causal mask built in NumPy; lower-triangular result plotted; np.triu check on GPT-2 attention
- ☐ Padding-mask demonstration and padding vs causal mask table
- ☐ Decoder and cross-attention Markdown; three model families explained
- ☐ t5-small translation on 3 sentences and summarisation of one real paragraph
- ☐ Cross-attention heatmap from t5-small (output tokens vs input tokens) with alignment discussion
- ☐ Manual greedy decoding loop on GPT-2 identical to model.generate(do_sample=False)
- ☐ Five decoding strategies compared in a table (greedy, beam, two temperatures, top-k + top-p); seeds varied for sampling
- ☐ KV-cache timing with use_cache=True vs False; speed-up reported
- ☐ Model-family comparison table (DistilBERT, GPT-2, T5-small)
- ☐ Pretrained sentiment model evaluated on 200 IMDB reviews: accuracy, precision, recall, F1, confusion matrix
- ☐ Truncation study at 64 / 128 / 256 / 512 tokens with time; number of reviews over 512 tokens reported
- ☐ Forward-pass time vs input length plotted for 32 to 512 tokens; quadratic cost discussed honestly
- ☐ Hallucination check: 3 factual prompts graded correct / partly correct / wrong
- ☐ text_toolkit() returns sentiment, summary and French translation; tested on 3 real passages; empty and long inputs handled
- ☐ Reflection section 'What I learned about Transformers' written
- ☐ Every task begins with a Markdown cell; all plots have titles and labels
- ☐ Kernel > Restart & Run All completed with zero errors; notebook exported to HTML

- ☐ requirements.txt committed with the library versions actually used
- ☐ Video: face (PiP) + screen, 5–10 min, MP4/MOV, named DL_PR6_YourName_GRID.mp4
- ☐ Video explains the 7 core concepts and includes a live text_toolkit() demo on a new passage
- ☐ GitHub: public repo with DL_PR6.ipynb + DL_PR6.html + /plots + requirements.txt
- ☐ README.md: exact Hugging Face model IDs, concept paragraphs, plots with captions, evaluation results, measured disadvantages, video link
- ☐ 3+ meaningful Git commits with descriptive messages
- ☐ Video link pasted in README.md under the Video section

© 2026 Red & White Skill Education. All Rights Reserved.